

PART 1 — The Problem Spark Solves

Big Data Problem

Traditional systems cannot process large datasets efficiently.

Example dataset sizes:

Data Type	Example Size
CSV logs	200 GB
Website clickstream	2 TB/day
IoT sensors	5 TB/day
Financial transactions	Petabytes

A single machine cannot handle this efficiently.

So we distribute computation across many machines.

Distributed Processing Analogy

Imagine sorting **1 billion exam papers**.

Approach 1 — One person

1 person sorting all papers

Time required = months

Approach 2 — 100 people

Divide papers into 100 piles

Each person sorts one pile

Time required = minutes

Spark follows the **second model**.

PART 2 — Evolution of Big Data Processing

Step 1: Traditional Databases

Used for structured data.

Limitations:

- Cannot handle TB/PB scale
- Vertical scaling expensive

Step 2: Hadoop MapReduce

Introduced distributed computing.

Workflow:

Read → Map → Shuffle → Reduce → Write

But MapReduce had problems:

Issue	Explanation
Disk heavy	Writes intermediate results to disk
Slow	Many disk reads/writes
Complex code	Hard to write jobs

Step 3: Apache Spark

Spark solves MapReduce problems.

Key improvement:

In-memory computation

Instead of writing intermediate data to disk, Spark keeps it in RAM.

Spark can be **up to 100x faster than MapReduce** due to in-memory processing.

Spark

PART 3 — What is Apache Spark?

Definition

Apache Spark is a **distributed data processing engine designed for speed, scalability, and flexibility**.

Spark

It processes large datasets across multiple machines.

Spark Characteristics

Feature	Explanation
Distributed	Runs on cluster of machines
In-memory processing	Faster computation
Fault tolerant	Recovers lost data
Multi-language support	Python, Scala, Java, R
Unified analytics engine	Batch + streaming + ML

PART 4 — Spark Ecosystem

Spark is not just one tool.

It is an ecosystem.

Component	Purpose
Spark Core	Base engine
Spark SQL	Structured data processing
Spark Streaming	Real-time streams
MLlib	Machine learning
GraphX	Graph analytics

These components are shown in the Spark document as the **Spark ecosystem modules**.

Spark

PART 5 — Spark vs Hadoop MapReduce

Feature	MapReduce	Spark
Processing	Disk based	In memory
Speed	Slow	Fast
Code	Complex	Simple
Workloads	Batch only	Batch + Streaming + ML
Iterative algorithms	Poor	Excellent

Example workload:

Customer purchase analysis

MapReduce:

Job1 → write to disk
Job2 → read from disk
Job3 → write again

Spark:

Load once → process in memory

PART 6 — Spark Architecture

Now explain **how Spark actually runs jobs**.

Major components:

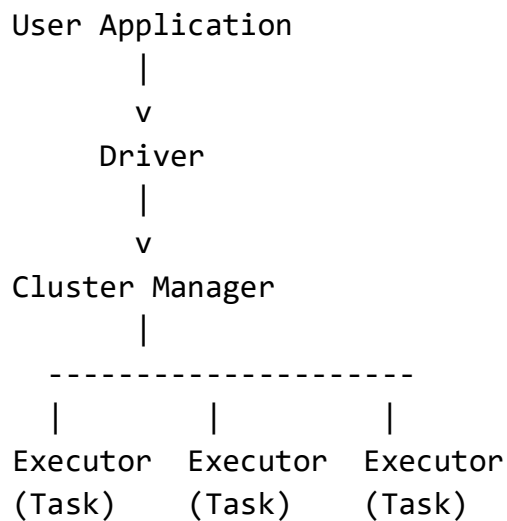
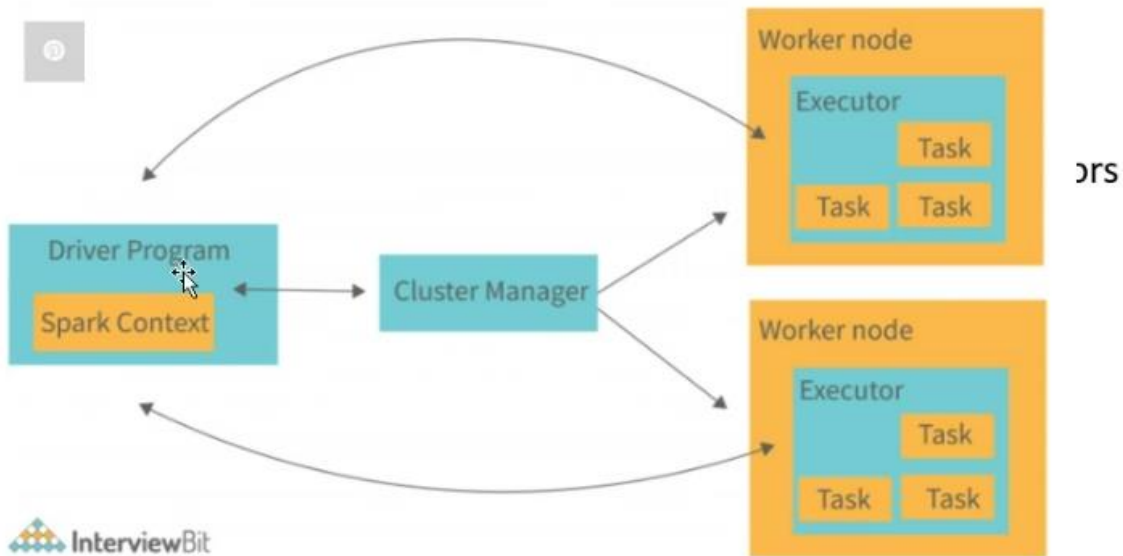
Component	Role
Driver	Controls application
Cluster Manager	Allocates resources
Executors	Run tasks
Tasks	Small units of work

These components form the core Spark architecture.

Spark

Spark Architecture Diagram (Conceptual)

Spark Architecture



Component Explanation

Driver Program

The driver:

- Runs the main Spark application
- Builds execution plan
- Sends tasks to workers

Example:

```
spark.read.csv()  
df.filter()  
df.groupBy()  
df.show()
```

Driver converts this into execution tasks.

Executors

Executors run on worker machines.

Responsibilities:

- Execute tasks
- Store data in memory
- Return results

Tasks

Tasks are **smallest units of work**.

Example:

Dataset:

100 million rows

Split into:

20 partitions

Spark creates:

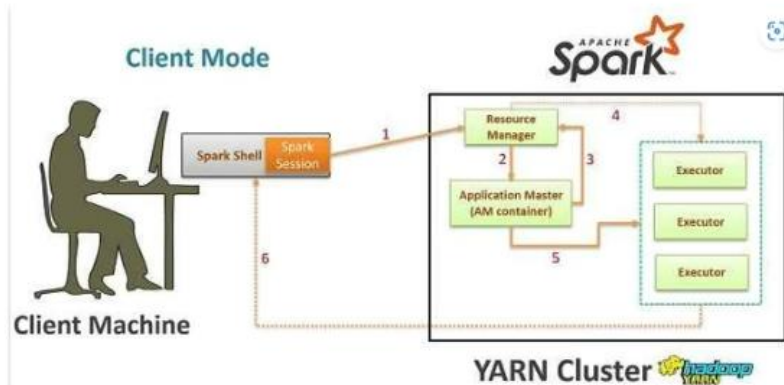
20 tasks

Each task processes one partition.

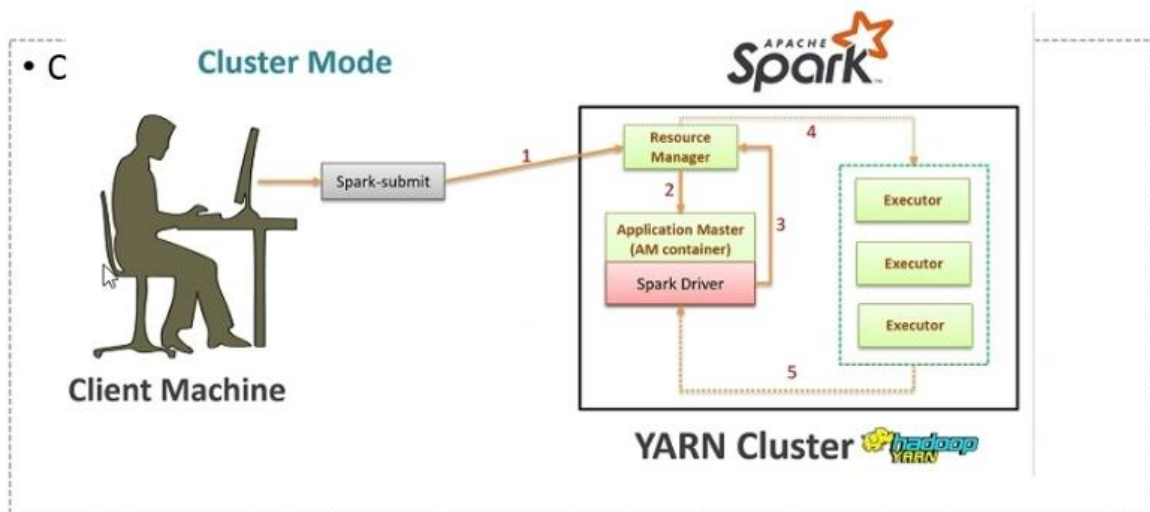
PART 7 — Spark Application Lifecycle

Client Modes

- Driver program runs on the client machine.
- Client machines needs to be running till you get the output



Cluster Mode



Execution steps:

- 1 User submits job
- 2 Driver starts

- 3 Cluster manager allocates resources
- 4 Executors start
- 5 DAG created
- 6 Tasks executed
- 7 Results returned

These steps form the Spark execution lifecycle described in the document.

Spark

PART 8 — DAG (Directed Acyclic Graph)

This is **one of the most important Spark concepts**.

What is a DAG?

Spark builds a **logical execution plan** called DAG.

It represents transformations and dependencies.

Spark executes only when an **action** occurs.

Spark

Example

PySpark code:

```
df = spark.read.csv("data.csv")
df2 = df.filter("age > 30")
df3 = df2.groupBy("city").count()
df3.show()
```

Execution behavior:

Step	Type
read	Transformation
filter	Transformation
groupBy	Transformation
show	Action

Spark records transformations but **does not execute immediately**.

Execution starts only when `.show()` runs.

PART 9 — Lazy Evaluation

Spark uses **lazy execution**.

Meaning:

Transformations are not executed immediately.

Spark waits until an **action is called**.

Benefits:

Benefit	Explanation
Optimization	Spark optimizes plan
Less computation	Avoid unnecessary steps
Better scheduling	Efficient resource use

PART 10 — Transformations vs Actions

This concept appears throughout Spark programming.

Types of function in spark

- Transformation :
 - lazy.
 - If a function returns you a new RDD from existing RDD
- Actions :
 - Eager.
 - It returns data in any form.
 - Output data is brought to driver node and result is given as output.



Transformations

Operations that **return a new dataset**.

Examples:

```
map()
filter()
select()
groupBy()
join()
```

Transformations are **lazy**.

Spark

Actions

Actions trigger computation.

Examples:

```
count()
collect()
show()
write()
```

Actions start execution.

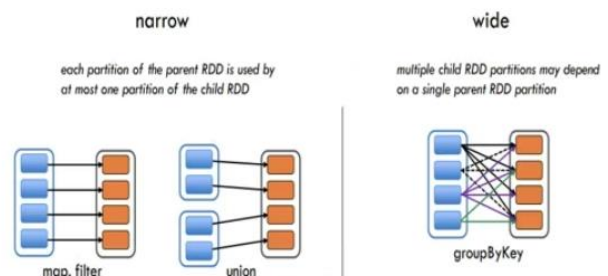
Spark

PART 11 — Narrow vs Wide Transformations

Critical for performance understanding.

Types of transformations in spark

- **Narrow :**
 - On same node. NO shuffling involved.
 - One parent(input) and one child (output)
- **Wide :**
 - Shuffling is involved because the data is distributed and needs to be collected at one executor.
 - One child partition is dependent on multiple parent



Narrow Transformations

Data stays in the same partition.

No network transfer.

Examples:

map
filter
select
union

These are fast operations.

Wide Transformations

Data moves between partitions.

This causes **shuffle**.

Examples:

groupBy
join
distinct
reduceByKey

Wide transformations are expensive because data moves across machines.

PART 12 — RDD vs DataFrame

Spark provides multiple APIs.

RDD

RDD stands for **Resilient Distributed Dataset**.

Characteristics:

- Low level API
- Immutable distributed collection
- Functional programming model

Example:

```
rdd.map()  
rdd.filter()
```

RDDs provide full control but require more code.

Spark

DataFrame

DataFrame is a higher level abstraction.

Similar to:

- SQL table
- Pandas dataframe

Advantages:

Benefit	Explanation
Less code	Simple API
Optimized	Catalyst optimizer
Faster	Tungsten execution engine

DataFrames are recommended for most workloads.

Spark

PART 13 — Spark SQL

Spark allows running SQL queries on large datasets.

Example workflow:

```
Load data
Register view
Run SQL
```

Example:

```
df.createOrReplaceTempView("employees")

spark.sql("""
SELECT department, AVG(salary)
FROM employees
GROUP BY department
""")
```

Spark SQL allows SQL users to work with big data easily.

Spark

PART 14 — Window Functions

Window functions allow calculations **without collapsing rows**.

Example use cases:

- Ranking employees
- Running totals
- Moving averages

Example concept:

```
ROW_NUMBER()
RANK()
```

LAG()
LEAD()

Window functions operate on partitions of data.

Spark

PART 15 — Performance Concepts

Spark performance depends on several factors.

Caching

Caching stores intermediate results in memory.

Example:

```
df.cache()
```

Useful when data reused multiple times.

Partitioning

Parallelism depends on partitions.

Too few partitions → slow

Too many partitions → overhead

Example:

```
df.repartition(10)
```

What is a Partition?

A **partition** is a logical chunk of data.

Spark splits large datasets into partitions so that **multiple executors can process them in parallel**.

Example dataset:

Total Rows	Partitions
1,000,000 rows	10 partitions
Each partition	100,000 rows

Spark will create:

10 tasks

Each task processes one partition.

This is how Spark achieves **parallel processing**.

Why Partitions Matter

Partition count determines **parallelism**.

Scenario	Result
Too few partitions	Cluster resources unused
Too many partitions	Scheduling overhead
Balanced partitions	Best performance

Example:

Cluster:

4 worker nodes

8 CPU cores

If partitions = 2

Only 2 tasks run → CPUs idle

If partitions = **1000**

Too many tasks → scheduling overhead

Optimal partitioning allows **maximum parallelism without overhead**.

Repartition vs Coalesce

Spark provides two main functions to change partition counts:

`repartition()`

`coalesce()`

These are used for **partition tuning and performance optimization**.

1 Repartition

Definition

`repartition()` **redistributes data across partitions using shuffle**.

It can:

- increase partitions
- decrease partitions
- rebalance data

Example:

```
df.repartition(10)
```

This redistributes the dataset into **10 partitions**.

What Happens Internally

Spark performs a **full shuffle**.

Meaning:

All data moves across cluster nodes

Example before repartition:

Partition1 → 90% data

Partition2 → 10% data

After repartition:

Partition1 → 50%

Partition2 → 50%

Balanced partitions improve parallel processing.

When to Use Repartition

Use `repartition()` when:

Scenario	Reason
Increasing partitions	Improve parallelism
Fixing skewed data	Balance partitions
Preparing for joins	Distribute join keys
Before heavy operations	Avoid bottlenecks

Example Scenario

Dataset:

```
sales_df
```

Current partitions:

```
2
```

Cluster capacity:

```
16 cores
```

Better configuration:

```
sales_df.repartition(16)
```

Now Spark can run **16 parallel tasks**.

2 Coalesce

Definition

`coalesce()` **reduces partitions without full shuffle.**

Example:

```
df.coalesce(4)
```

This merges existing partitions into **4 partitions**.

What Happens Internally

Unlike `repartition`:

No full shuffle

Spark simply merges partitions.

Example:

Before:

Partition1
Partition2
Partition3
Partition4
Partition5
Partition6
Partition7
Partition8

After:

Partition1
Partition2
Partition3
Partition4

Partitions are **combined**, not redistributed.

When to Use Coalesce

Use `coalesce()` when:

Scenario	Reason
Reducing partitions	Avoid shuffle
Writing output files	Reduce file count
Final output stage	Optimize storage

Example Scenario

After processing:

200 partitions

Writing output:

200 files created

Instead:

```
df.coalesce(10).write.parquet(...)
```

Now Spark writes:

10 output files

Repartition vs Coalesce Comparison

Feature	Repartition	Coalesce
Shuffle	Yes	No (usually)
Increase partitions	Yes	No
Decrease partitions	Yes	Yes
Performance	Slower	Faster
Use case	balancing data	reducing partitions

Before Repartition

Partition1 → 80GB

Partition2 → 20GB

Workload imbalance.

After Repartition

Partition1 → 50GB

Partition2 → 50GB

Balanced workload.

Coalesce Example

Before writing:

200 partitions

Output files:

200 files

Using:

```
df.coalesce(10)
```

Output files:

10 files

Important Performance Insight

repartition = expensive

coalesce = cheaper

Why?

Because **repartition triggers shuffle**.

Shuffles involve:

- network transfer
- disk spill
- serialization

File Formats

Use columnar formats:

Format	Advantage
Parquet	Columnar + compressed
ORC	Optimized for analytics
CSV	Slow
JSON	Slow

PART 16 — Spark Internals

Two major internal optimizers:

Catalyst Optimizer

Optimizes query plan.

Example:

```
SELECT name FROM employees WHERE age > 30
```

Catalyst rewrites query for efficient execution.

Tungsten Engine

Handles:

- memory management
- CPU optimization
- code generation

Together they make Spark very fast.

Conceptual Exercise

Exercise 1

Dataset:

100GB sales data

Operations:

filter
groupBy
join
write

Question:

- Which operations cause shuffle?
- How many stages will be created?

Step 1 — Identify Transformations

Operation	Type
filter	Narrow transformation
groupBy	Wide transformation
join	Wide transformation
write	Action

Step 2 — Which Operations Cause Shuffle?

Shuffle happens during **wide transformations**.

Operation	Shuffle?	Reason
filter	No	Data stays in same partition
groupBy	Yes	Data must be redistributed by key
join	Yes	Data must align across partitions
write	No	Writes existing partitions

Spark creates **a new stage for every shuffle boundary**.

Rule:

Number of stages = number of wide transformations + 1

Wide transformations here:

groupBy
join

So:

2 wide transformations

Stages:

Stage 1 → filter
Stage 2 → groupBy
Stage 3 → join + write

Final Answer

3 stages

Exercise 2

If dataset has:

10 partitions

How many tasks will run?

Answer:

10 tasks

In Spark:

1 partition = 1 task

So:

10 partitions → 10 tasks

These tasks can run **in parallel depending on available CPU cores**.

Example:

Partitions	Tasks
5	5
10	10
50	50

Final Answer

10 tasks

Conceptual Lab

Lab Scenario

Retail company analyzing orders.

Data columns:

order_id
customer_id
product_id

price
city
date

Questions:

- 1 Find total sales per city
 - 2 Find top selling products
 - 3 Find repeat customers
- Which transformations used?
 - Where shuffle occurs?
 - Which action triggers execution?

Find total sales per city

Required Transformations

groupBy
aggregation (sum)

Example logical steps:

```
groupBy(city)  
sum(price)
```

Shuffle?

Yes.

Why?

Because Spark must **bring all rows for the same city together**.

Shuffle operation

```
groupBy(city)
```

Find top selling products

Steps:

```
groupBy(product_id)
sum(price)
orderBy(desc(total_sales))
limit(10)
```

Transformations involved

Operation	Type
groupBy	Wide
sum	Aggregation
orderBy	Wide
limit	Narrow

Shuffle occurs at

groupBy
orderBy

Because both require **data redistribution**.

Find repeat customers

Definition:

Customers with multiple orders

Logical steps:

```
groupBy(customer_id)
count(order_id)
filter(count > 1)
```

Transformations

Operation	Type
groupBy	Wide

count	Aggregation
filter	Narrow

Shuffle occurs at

```
groupBy(customer_id)
```